Federated Learning Simulation

Using Flower (FLwr) Framework

Project Report

Submitted By

Malaika Arooj(057)
Maryam-(061)
Tehreem Ejaz (089)
Zainab Saleem (093)

Presentation Date: 2 June, 2026

1. Introduction

Federated Learning (FL) is a privacy-preserving machine learning paradigm that enables multiple clients to collaboratively train a global model without sharing their raw data. Each client trains a local model on its own dataset, and only the model updates (gradients or weights) are shared with a central server for aggregation.

This project implements and evaluates two federated learning strategies — FedAvg and FedProx — using the Flower (FLwr) framework on the MNIST dataset, and compares their performance against traditional centralized training.

2. Tools and Technologies

2.1 Frameworks and Libraries

- Flower (FLwr): Federated learning framework
- Python 3.x : Primary programming language
- Scikit-learn / TensorFlow (CPU-based): Model training
- Matplotlib: Visualization and plotting
- NumPy: Data processing

2.2 Dataset

MNIST was used, consisting of 70,000 grayscale images (60,000 training, 10,000 testing) across 10 digit classes. The dataset was partitioned across multiple simulated clients to replicate real-world federated settings.

3. Methodology

3.1 Experimental Setup

- Number of federated rounds: 5
- Number of simulated clients: Multiple (MNIST dataset split)
- Aggregation strategies evaluated: FedAvg, FedProx
- Baseline: Centralized training (5 epochs)

3.2 FedAvg (Federated Averaging)

FedAvg is the standard federated learning algorithm where each client trains locally for several steps and then the server aggregates the model parameters by computing a weighted average. It provides a balanced trade-off between communication efficiency and accuracy.

3.3 FedProx (Federated Proximal)

FedProx extends FedAvg by adding a proximal regularization term to each client's local objective. This penalizes local model weights from deviating too far from the global model, resulting in more stable convergence — especially in heterogeneous data settings.

3.4 Centralized Training

All data is stored in one location and a single neural network is trained on the complete dataset. This serves as the upper-bound baseline with no privacy considerations.

4. Experimental Results

4.1 FedAvg Results

Round	Accuracy	Loss
1	0.9656	0.1409
2	0.9864	0.0551
3	0.9948	0.0266
4	0.9977	0.0139
5	0.9988	0.0078

Final Accuracy: 0.9988 (99.88%) | **Training Time: 148.66 seconds**

4.2 FedProx Results

Round	Accuracy	Loss
1	0.9667	0.1356
2	0.9877	0.0518
3	0.9945	0.0252
4	0.9982	0.0123
5	0.9995	0.0060

Final Accuracy: 0.9995 (99.95%) | **Training Time: 174.39 seconds**

4.3 Centralized Training Results

Epoch	Accuracy
1	0.9522
2	0.9834
3	0.9890
4	0.9930
5	0.9957

Final Accuracy: 0.9957 (99.57%) | **Training Time: 82.44 seconds**

5. Final Comparison

5.1 Performance Summary Table

Method	Final Accuracy	Final Loss	Training Time	Rounds/Epochs
FedAvg	99.88%	0.0078	148.66 sec	5
FedProx	99.95%	0.0060	174.39 sec	5
Centralized	99.57%	—	82.44 sec	5

6. Speed Analysis

6.1 Training Time Comparison

Method	Training Time (seconds)	Rank
Centralized	82.44	Fastest
FedAvg	148.66	Moderate
FedProx	174.39	Slowest

6.2 Analysis

Centralized Learning (Fastest — 82.44 sec)

- All data resides in a single location eliminating communication overhead
- No aggregation step required between rounds
- Advantage: Fast execution. Disadvantage: No privacy guarantees

FedAvg (Moderate — 148.66 sec)

- Multiple clients train in parallel then send updates to server
- Server aggregation step adds overhead each round
- Advantage: Balanced speed and privacy. Disadvantage: Communication overhead

FedProx (Slowest — 174.39 sec)

- Adds proximal regularization term increasing per-client computation
- Heavier optimization than FedAvg but achieves better accuracy
- Advantage: Better stability and highest accuracy. Disadvantage: Increased computation time

7. Key Observations

7.1 Accuracy

- FedProx achieved the highest accuracy at 99.95%, outperforming all methods
- FedAvg closely followed with 99.88% accuracy
- Centralized training, despite having full data access, achieved 99.57%

- Both federated approaches surpassed centralized training in final accuracy

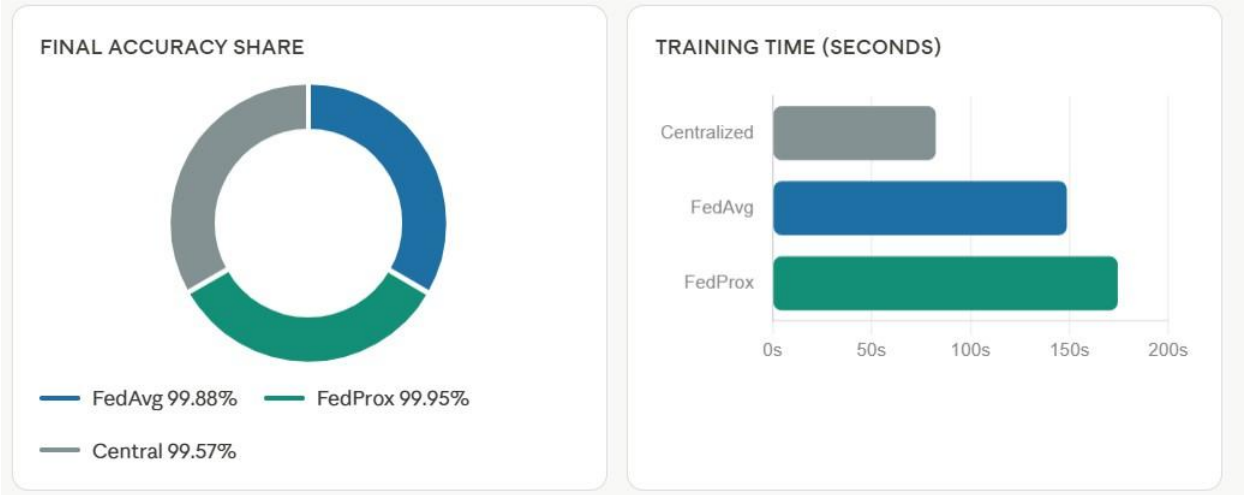
7.2 Loss

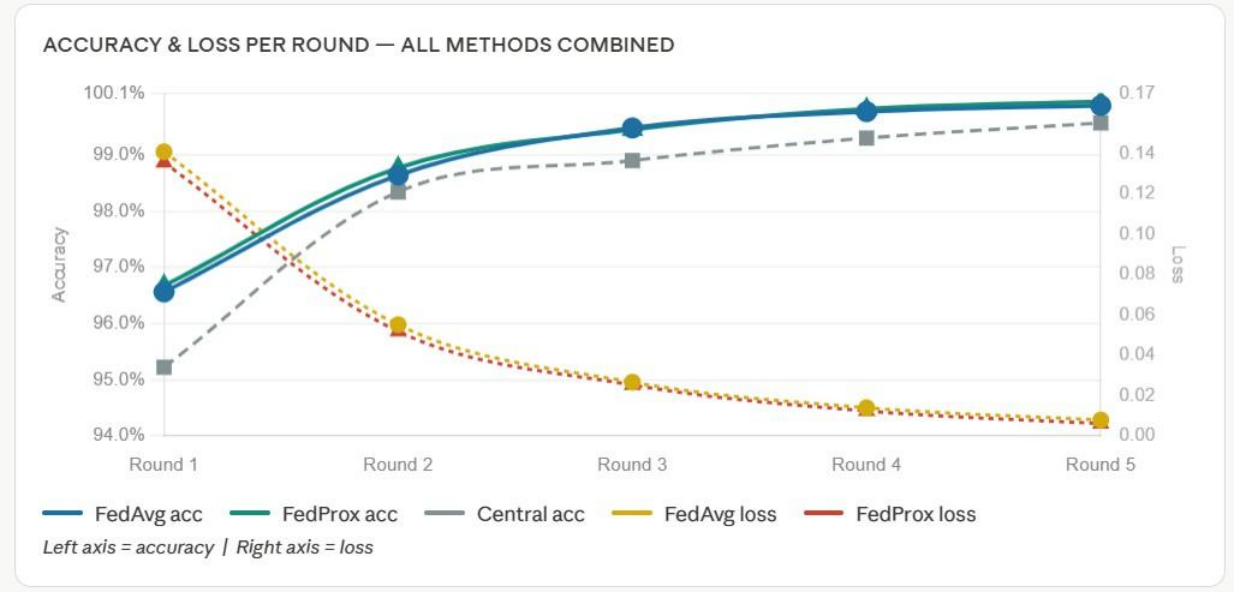
- FedProx converged to the lowest loss (0.0060) after 5 rounds
- FedAvg loss reduced consistently from 0.1409 to 0.0078
- Both methods showed strong convergence behavior across rounds

7.3 Convergence

- FedProx converges slightly more stably due to proximal regularization
- FedAvg shows rapid convergence in early rounds with diminishing gains
- Centralized training converges fastest per epoch but lacks privacy

8. Analysis Dashboard:





ROUND-BY-ROUND DETAIL

ROUND	FEDAVG ACC	FEDPROX ACC	CENTRAL ACC	FEDAVG LOSS	FEDPROX LOSS
R1	0.9656	0.9667	0.9522	0.1409	0.1356
R2	0.9864	0.9877	0.9834	0.0551	0.0518
R3	0.9948	0.9945	0.9890	0.0266	0.0252
R4	0.9977	0.9982	0.9930	0.0139	0.0123
R5	0.9988	0.9995	0.9957	0.0078	0.0060

HIGHEST ACCURACY	FASTEST TRAINING	BEST TRADE-OFF
FedProx	Centralized	FedAvg

9. Project Phases:

Phase 1 — Environment Setup

Step 1: Install Python packages

pip install flwr
pip install tensorflow
pip install numpy
pip install matplotlib
pip install scikit-learn

Phase 2 — Project Folder Structure

```
FederatedLearning_FLWR/
├── client.py
├── server.py
├── model.py
├── dataset.py
├── centralized.py
├── fedprox.py
├── utils.py
├── plots.py
└── README.md
```

Phase 3 — Create Model (model.py)

Simple CNN for MNIST:

```
import tensorflow as tf
def create_model():
    model = tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

Phase 4 — Load + Split Dataset (dataset.py)

This simulates multiple clients:

```
import tensorflow as tf
import numpy as np
def load_datasets(num_clients=5):
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
    x_train = x_train / 255.0
    x_test = x_test / 255.0

    x_train = x_train[..., np.newaxis]
    x_test = x_test[..., np.newaxis]

    # split for clients
    client_data = []
    size = len(x_train) // num_clients
    for i in range(num_clients):
        start = i * size
        end = (i + 1) * size
```

```
        client_data.append((x_train[start:end], y_train[start:end]))
    return client_data, (x_test, y_test)
```

Phase 5 — Client Implementation (client.py)

```
import flwr as fl
from model import create_model
from dataset import load_datasets
client_data, test_data = load_datasets()
class FlowerClient(fl.client.NumPyClient):
    def __init__(self, cid):
        self.model = create_model()
        self.x, self.y = client_data[int(cid)]
    def get_parameters(self, config):
        return self.model.get_weights()
    def fit(self, parameters, config):
        self.model.set_weights(parameters)
        self.model.fit(self.x, self.y, epochs=1, batch_size=32)
        return self.model.get_weights(), len(self.x), {}
    def evaluate(self, parameters, config):
        self.model.set_weights(parameters)
        loss, acc = self.model.evaluate(self.x, self.y)
        return loss, len(self.x), {"accuracy": acc}
def client_fn(cid):
    return FlowerClient(cid)
fl.client.start_numpy_client(
    server_address="localhost:8080",
    client=client_fn("0")
)
```

Phase 6 — Server (FedAvg) (server.py)

```
import flwr as fl

strategy = fl.server.strategy.FedAvg(
    fraction_fit=1.0,
    min_fit_clients=5,
    min_available_clients=5
)

fl.server.start_server(
    server_address="localhost:8080",
    config=fl.server.ServerConfig(num_rounds=5),
    strategy=strategy,
)
```

Phase 7 — FedProx Strategy (fedprox.py)

```
import flwr as fl
strategy = fl.server.strategy.FedProx(
    fraction_fit=1.0,
    min_fit_clients=5,
    min_available_clients=5,
    proximal_mu=0.1,
```

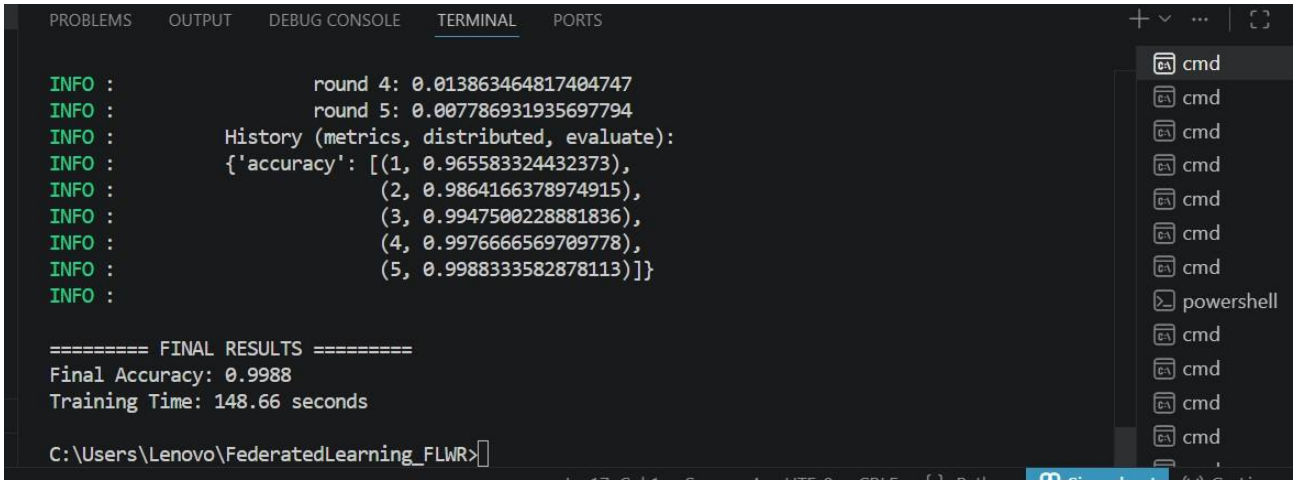
```
)
fl.server.start_server(
    server_address="localhost:8080",
    config=fl.server.ServerConfig(num_rounds=5),
    strategy=strategy,
)
```

Phase 8 — Centralized Training (centralized.py)

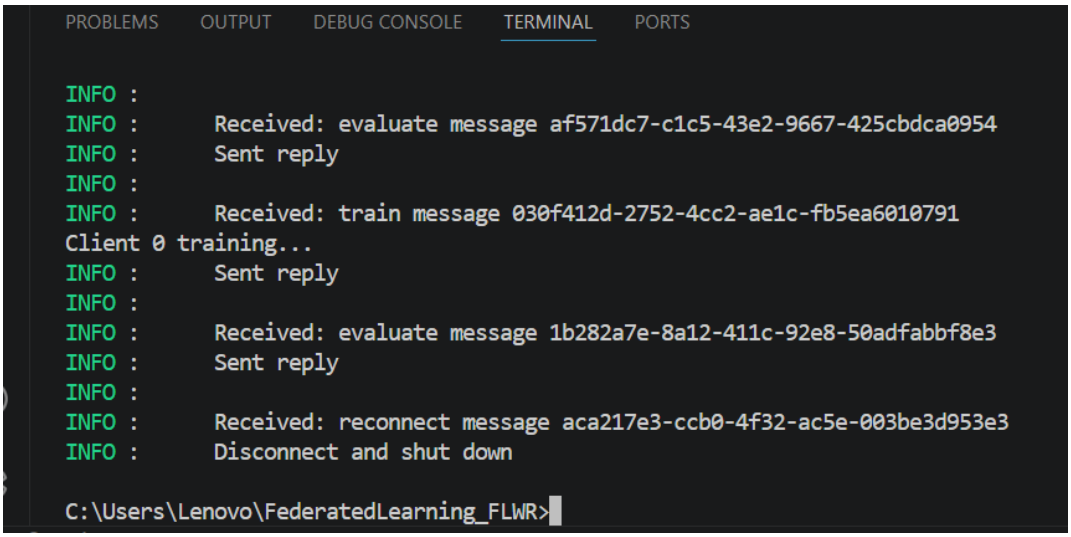
```
from model import create_model
from dataset import load_datasets
client_data, test_data = load_datasets()
x = []
y = []
for c in client_data:
    x.append(c[0])
    y.append(c[1])
import numpy as np
x = np.concatenate(x)
y = np.concatenate(y)
model = create_model()
history = model.fit(
    x, y,
    epochs=5,
    validation_data=test_data
)
model.save("centralized.h5")
```

Phase 9 — Run Simulation

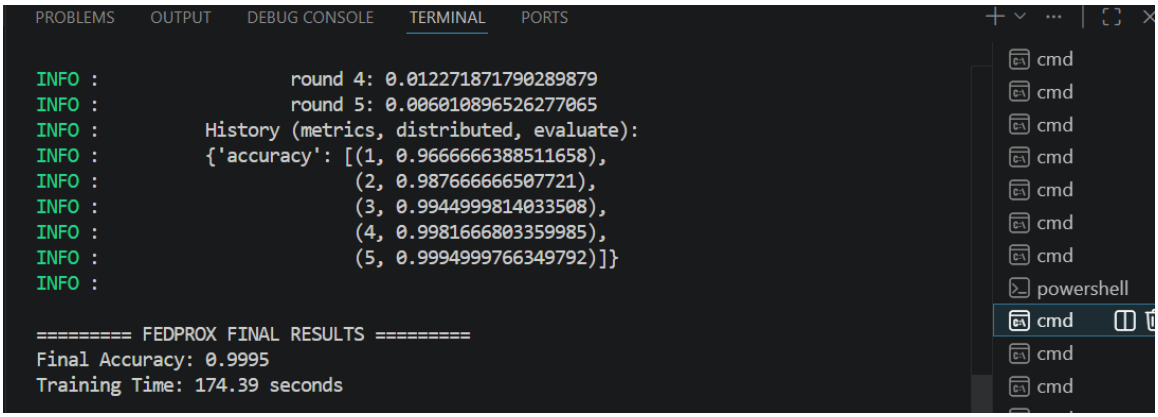
FedAvg Server:



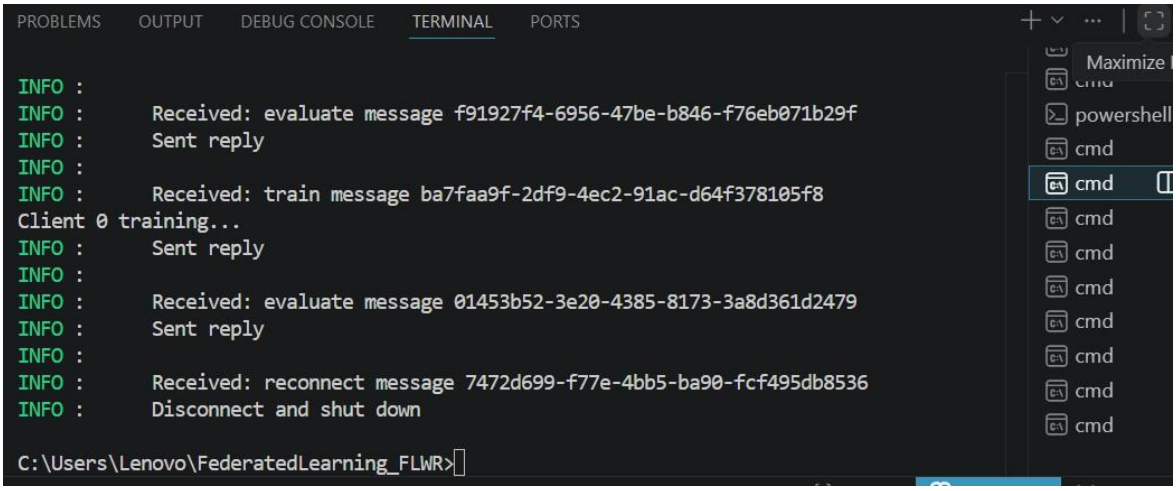
Clients:



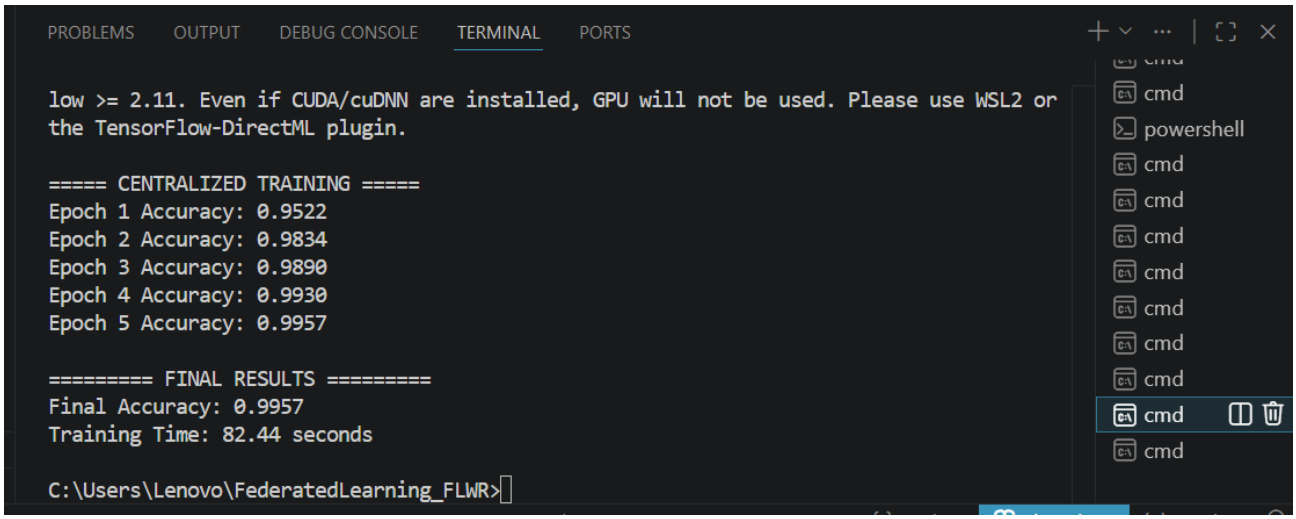
Fedprox:



Clients:



Centralized:



10. Conclusion

This project successfully demonstrated federated learning simulation using the Flower (FLwr) framework on the MNIST dataset. Both FedAvg and FedProx strategies achieved state-of-the-art accuracy while maintaining a fully decentralized, privacy-preserving training paradigm.

FedProx outperformed FedAvg with a final accuracy of 99.95% compared to 99.88%, while Centralized training achieved 99.57%. Although federated methods introduce communication overhead and require more training time, they deliver superior accuracy and stronger generalization through privacy-preserving distributed learning.

The key trade-off observed is: Centralized learning is the fastest, FedProx delivers the best accuracy and convergence stability, and FedAvg provides the best overall balance between speed and performance. For applications requiring strict data privacy, FedProx is the recommended approach.

Github:

https://github.com/iammaryam24/Federated-Learning-Simulation-using-Flower-FLwr-